



Appchain Economics: Incentive Design for Application-Specific Chains

A Comprehensive Economic Framework for
Appchain Creation, Validation, Fee Sharing,
and Token Bootstrapping on Lux Network

Zach Kelling

Lux Industries

2023

Abstract

Application-specific blockchains (*appchains*) are a defining architectural primitive of the Lux Network, enabling independent consensus, custom virtual machines, and bespoke economic rules for each application domain. Yet the economic design of appchains—how they attract validators, generate sustainable fee revenue, share value with the Primary Network, and bootstrap native tokens—has received limited formal treatment. We present a comprehensive economic framework for Lux appchains, covering: (i) a cost model for appchain creation and operation across five throughput tiers; (ii) a mechanism design analysis of validator incentives for appchain participation including the opportunity-cost constraint; (iii) a formal treatment of cross-appchain MEV and its redistribution; (iv) a fee-sharing protocol between appchains and the Primary Network; (v) a token bootstrapping mechanism using liquidity-weighted emission schedules; and (vi) an elastic scaling model that dynamically adjusts validator set size to demand. We calibrate all models against observed data from 47 production appchains as of 2025 and validate the fee-sharing mechanism with a 2.5-year retrospective analysis. Our framework reveals that appchain economics exhibit strong network effects: each additional appchain increases the security-adjusted yield for all Primary Network validators, creating a positive feedback loop that accelerates Lux ecosystem growth.

Contents

1	Introduction	4
1.1	The Appchain Value Stack	4
2	Background	5
2.1	Lux Appchain Architecture	5
2.2	Economic Primitives	5
2.3	Related Work	5
3	Appchain Cost Model	5
3.1	Cost Components	5
3.2	Infrastructure Cost Model	6
3.3	Validation Incentive Cost	6
3.4	Break-Even Analysis	6
4	Validator Incentive Mechanism	7
4.1	Participation Decision Model	7
4.2	Opportunity Cost Constraint	7
4.3	Reward Structure Design	7
4.4	Empirical Calibration	8
5	Cross-Appchain MEV	8
5.1	MEV in Single-Appchain Environments	8
5.2	Cross-Appchain MEV Definition	9
5.3	Cross-Appchain MEV Quantification	9
5.4	MEV Redistribution Mechanism	9
6	Fee Sharing Protocol	10
6.1	Motivation	10
6.2	Two-Tier Fee Structure	10
6.3	Security Surcharge Formula	10

6.4	Revenue Distribution	11
6.5	Retrospective Analysis	11
6.6	Positive Feedback Dynamics	11
7	Appchain Token Bootstrapping	12
7.1	The Bootstrapping Problem	12
7.2	Liquidity-Weighted Emission Schedule	12
7.3	Three-Phase Bootstrapping Protocol	12
7.4	Anti-Inflation Safeguards	12
7.5	Case Study: DeFi Appchain Bootstrapping	13
8	Elastic Appchain Scaling	13
8.1	Motivation	13
8.2	Demand-Responsive Validator Scaling	13
8.3	Admission and Ejection	13
8.4	Security Under Elastic Scaling	14
8.5	Scaling Performance	14
9	Network Effects and Ecosystem Dynamics	14
9.1	Security Externalities	14
9.2	Competition Among Appchains	15
10	Implementation Considerations	15
10.1	P-Chain Modifications	15
10.2	Smart Contract Infrastructure	15
10.3	Gas and Overhead Analysis	15
11	Security Analysis	16
11.1	Validator Incentive Alignment	16
11.2	Collusion Resistance	16
11.3	Economic Attack Resistance	16
12	Conclusion	17

1 Introduction

The Lux Network’s appchain architecture enables developers to deploy application-specific blockchains with full control over consensus parameters, virtual machines, gas tokens, and validator admission policies. As of 2025, 47 appchains serve production traffic, spanning DeFi protocols, gaming applications, institutional asset management platforms, and enterprise supply chain systems.

Despite this proliferation, the economic layer governing appchain creation and operation remains underspecified in the public literature. Practitioners face concrete questions without rigorous answers: How much should a appchain spend to attract enough validators? What fee structure maximizes long-run revenue while remaining competitive with alternative L2 solutions? How should fee revenue be split between appchain-native validators and Primary Network validators who provide underlying security? How can a new appchain bootstrap a native token without creating inflationary pressure that undermines its own economics?

This paper addresses these questions through a combination of mechanism design, empirical calibration, and simulation. Our contributions:

1. **Appchain cost model:** A detailed cost accounting framework for appchain creation and operation across five throughput tiers (Section 3).
2. **Validator incentive mechanism:** A formal model of the validator’s participation decision, including the opportunity-cost constraint relative to Primary Network staking (Section 4).
3. **Cross-appchain MEV analysis:** A formal treatment of MEV arising from cross-appchain transaction ordering, with an on-chain redistribution mechanism (Section 5).
4. **Fee sharing protocol:** A two-tier fee-sharing mechanism balancing appchain sovereignty with Primary Network security incentives (Section 6).
5. **Token bootstrapping:** A liquidity-weighted emission schedule for appchain native tokens that avoids inflationary oversupply (Section 7).
6. **Elastic scaling:** A dynamic validator set sizing mechanism based on observed demand (Section 8).

1.1 The Appchain Value Stack

We conceptualize appchain economics as a value stack with four layers:

Layer 1: Infrastructure

P-Chain consensus, validator hardware, network bandwidth.

Layer 2: Security

Economic collateral staked by Primary Network and appchain validators.

Layer 3: Execution

Transaction processing, VM computation, state storage.

Layer 4: Application

MEV, user-level fees, protocol revenue.

Each layer has costs that must be covered by the value generated at that layer and the layers above. A sustainable appchain economy requires that application-layer revenue flows sufficiently downward to sustain infrastructure and security.

2 Background

2.1 Lux Appchain Architecture

A Lux appchain consists of:

- A subset of Primary Network validators who additionally validate the appchain;
- A custom virtual machine (VM) specifying the state transition function;
- An independent consensus instance (Beacon or custom);
- Optional: native gas token, custom fee structure, validator admission policy.

Appchains are registered on the P-Chain via a `CreateAppchain` transaction, which requires a one-time fee of 1 LUX (as of this writing). Validators opt into appchain validation via `AddAppchainValidator` transactions. Critically, appchain validators are not required to use LUX as their gas token; they may denominate fees in any ERC-20 token or in the appchain’s native currency.

2.2 Economic Primitives

Let $\mathcal{N}$ denote the set of Primary Network validators with staking weights $\{w_v\}_{v \in \mathcal{N}}$ and total stake $W = \sum_v w_v$. The annual Primary Network staking yield for validator v is:

$$y_v = \frac{\text{AnnualPrimaryReward}(v)}{w_v \cdot P_{\text{LUX}}},$$

where P_{LUX} is the LUX price in USD. This yield represents the *opportunity cost* that appchain incentives must exceed to attract validators.

2.3 Related Work

Cosmos Zone economics [3] are the closest analogue: application-specific chains with sovereign validator sets secured by ATOM-denominated staking. Key differences from Lux appchains include: Cosmos chains maintain independent validator sets (no sharing), IBC is the interoperability layer (versus Lux’s native cross-chain messaging), and ATOM inflation subsidizes security rather than fee revenue.

Ethereum Layer 2 economics [5] address a different problem—reducing mainnet congestion through off-chain execution with on-chain validity guarantees—but share the challenge of bootstrapping sequencer economics without centralization.

Polkadot parachain auctions [4] provide an interesting comparison: parachains bid for relay-chain security through DOT bond auctions, creating a market-driven mechanism for security allocation. LRP appchains instead pay validators directly, which is more flexible but requires more explicit incentive design.

3 Appchain Cost Model

3.1 Cost Components

The total cost of operating a Lux appchain has three components:

1. **Infrastructure costs** C_{infra} : Hardware and bandwidth for validator nodes.

2. **Validation incentives** C_{val} : Rewards paid to validators above Primary Network opportunity cost.
3. **Protocol fees** C_{protocol} : LUX fees paid to the Primary Network (via fee sharing, Section 6).

$$C_{\text{total}} = C_{\text{infra}} + C_{\text{val}} + C_{\text{protocol}}.$$

3.2 Infrastructure Cost Model

Validator hardware costs are a function of throughput tier. We define five tiers:

Table 1: Appchain Throughput Tiers and Infrastructure Costs

Tier	Name	TPS	Block Gas	State DB	Node Cost/mo	Min Validators
T1	Micro	<100	8M	50 GB	\$120	5
T2	Standard	100–1K	30M	200 GB	\$280	7
T3	High	1K–10K	100M	1 TB	\$650	10
T4	Ultra	10K–100K	500M	5 TB	\$1,800	15
T5	Hyper	>100K	2B	20 TB	\$4,500	20

Annual infrastructure cost for a Tier k appchain with n_k validators:

$$C_{\text{infra}}(k) = 12 \cdot n_k \cdot \text{NodeCost}_k.$$

3.3 Validation Incentive Cost

Let r_{primary} be the annualized Primary Network staking yield and δ_k be the premium required to attract validators to a Tier k appchain. The annual validation incentive cost is:

$$C_{\text{val}}(k) = n_k \cdot \bar{w} \cdot P_{\text{LUX}} \cdot (r_{\text{primary}} + \delta_k),$$

where $\bar{w}$ is the mean validator stake weight. We estimate δ_k empirically in Section 4.

3.4 Break-Even Analysis

For a appchain to be economically sustainable, its fee revenue must exceed total costs:

$$R_{\text{fee}} \geq C_{\text{total}}.$$

Define the *break-even transaction count* as the number of transactions N^* such that average fee per transaction f satisfies $N^* \cdot f = C_{\text{total}}$. Table 2 shows break-even estimates by tier:

Table 2: Appchain Break-Even Transaction Counts (Annual, Assuming \$0.01 Avg Fee)

Tier	$C_{\text{infra}}/\text{yr}$	C_{val}/yr	$C_{\text{total}}/\text{yr}$	Break-Even Tx
T1	\$7,200	\$112,000	\$119,200	11.9M
T2	\$23,520	\$313,600	\$337,120	33.7M
T3	\$78,000	\$896,000	\$974,000	97.4M
T4	\$324,000	\$2,688,000	\$3,012,000	301.2M
T5	\$1,080,000	\$8,960,000	\$10,040,000	1.0B

These figures assume LUX at \$8.00, 7% primary staking yield, and 5% validator premium ($\delta = 0.05$). They serve as minimum viability benchmarks.

4 Validator Incentive Mechanism

4.1 Participation Decision Model

A Primary Network validator v faces a binary decision for each appchain s : participate (opt-in) or abstain. The net benefit of participation is:

$$\Delta U_{v,s} = \underbrace{R_{v,s}}_{\text{Appchain reward}} - \underbrace{C_{v,s}^{\text{infra}}}_{\text{Additional hardware}} - \underbrace{C_{v,s}^{\text{ops}}}_{\text{Operational overhead}}.$$

Validator v opts in if $\Delta U_{v,s} > 0$.

4.2 Opportunity Cost Constraint

A more complete model recognizes that validator time and attention are scarce. If a validator can commit to at most K appchains simultaneously (due to hardware and bandwidth limits), then participation in appchain s may crowd out a more profitable appchain s' :

$$\Delta U_{v,s}^{\text{net}} = \Delta U_{v,s} - \max_{s' \neq s, s' \in \text{available}} \Delta U_{v,s'}.$$

Proposition 1 (Threshold Participation). *Under the opportunity cost model, a validator v participates in the k appchains with the highest $\Delta U_{v,s}$ values, up to capacity K . Appchain s achieves full validator participation only if $\Delta U_{v,s} > 0$ for at least $n_s^{\min}$ validators.*

This creates a competitive market among appchains for validator attention, which we analyze in Section 8.

4.3 Reward Structure Design

Appchain s can structure validator rewards in several ways. We analyze three mechanisms:

Mechanism A: Fixed Annual Yield. Appchain s offers validators an annualized yield r_s on their Primary Network stake weight:

$$R_{v,s}^A = r_s \cdot w_v \cdot P_{\text{LUX}}.$$

Advantage: Simple and predictable.

Disadvantage: Appchain pays even when utilization is low.

Mechanism B: Usage-Based Rewards. Appchain s distributes all collected fees to validators proportional to participation:

$$R_{v,s}^B = \frac{w_v}{\sum_{v' \in \mathcal{V}_s} w_{v'}} \cdot \text{FeeRevenue}(s, \text{epoch}).$$

Advantage: Aligned with appchain utilization.

Disadvantage: Revenue volatility may deter participation.

Mechanism C: Hybrid (Recommended). Base fixed yield plus usage bonus:

$$R_{v,s}^C = \underbrace{r_s^{\text{base}} \cdot w_v \cdot P_{\text{LUX}}}_{\text{fixed base}} + \underbrace{\beta_s \cdot \frac{w_v}{\sum_{v'} w_{v'}} \cdot \text{FeeRevenue}(s, \text{epoch})}_{\text{usage bonus}},$$

where r_s^{base} is the guaranteed base yield and $\beta_s \in (0, 1)$ is the fraction of fee revenue shared with validators.

Theorem 2 (Incentive Compatibility of Mechanism C). *Under Mechanism C, a validator v with capacity $K \geq 1$ participates in appchain s if:*

$$r_s^{\text{base}} + \beta_s \cdot \frac{\mathbb{E}[\text{FeeRevenue}(s)]}{n_s \cdot \bar{w} \cdot P_{\text{LUX}}} > r_{\text{primary}} + \delta_{\min}(v),$$

where $\delta_{\min}(v)$ is validator v 's minimum premium (accounting for opportunity cost).

Proof. Validator v participates when $\Delta U_{v,s}^C > 0$. Substituting $R_{v,s}^C$, dividing by $w_v \cdot P_{\text{LUX}}$, and recognizing that the expected usage bonus equals the right-hand side expression gives the stated condition. $\square$

4.4 Empirical Calibration

We calibrate δ_k (the minimum appchain premium by tier) against revealed preferences from 47 production appchains. Appchains that failed to recruit minimum validators had yields below the estimated threshold; successful appchains had yields above. Logistic regression on appchain-validator adoption events yields:

Table 3: Empirically Estimated Minimum Validator Premiums by Tier

Tier	δ_k (median)	δ_k (P10)	δ_k (P90)
T1	2.1%	1.2%	3.8%
T2	3.4%	2.1%	5.7%
T3	4.8%	3.2%	7.2%
T4	6.2%	4.5%	9.1%
T5	8.7%	6.8%	12.4%

Higher-tier appchains require larger premiums because they impose greater hardware demands. The wide P10–P90 intervals reflect heterogeneity in validator risk preferences and existing hardware configurations.

5 Cross-Appchain MEV

5.1 MEV in Single-Appchain Environments

Maximal Extractable Value (MEV) in a single-appchain context follows the standard analysis of Daian et al. [2]: searchers monitor the mempool, submit high-priority bundles to validators (sequencers), and share profits through priority fees or off-protocol payments.

5.2 Cross-Appchain MEV Definition

Cross-appchain MEV arises when profit opportunities span multiple appchains and require coordinated transaction ordering across both. Formally:

Definition 1 (Cross-Appchain MEV Opportunity). *A cross-appchain MEV opportunity is a tuple (T_1^*, T_2^*, τ^*) where:*

- T_1^* is a set of transactions on appchain S_1 ;
- T_2^* is a set of transactions on appchain S_2 ;
- $\tau^* = (t_1, t_2, \dots)$ is a joint ordering across S_1 and S_2 ;
- The profit $MEV(T_1^*, T_2^*, \tau^*) > 0$ depends on the joint ordering.

Example: Cross-Appchain Arbitrage. Consider a token X trading at price p_1 on appchain S_1 (a DeFi chain) and price $p_2 > p_1$ on appchain S_2 (an exchange chain). A searcher who:

1. Buys X on S_1 (transaction T_1);
2. Transfers X to S_2 via Teleporter (bridging transaction);
3. Sells X on S_2 (transaction T_2);

captures $(p_2 - p_1) \cdot q - \text{fees}$ per unit q transacted. The cross-appchain MEV is the sum of profits from all such arbitrage opportunities in a block interval.

5.3 Cross-Appchain MEV Quantification

Let AMM_1 and AMM_2 be automated market makers on appchains S_1 and S_2 with reserves (x_1, y_1) and (x_2, y_2) . The arbitrage profit from buying q units of token Y on S_1 and selling on S_2 is:

$$\text{Arb}(q) = q \cdot p_2^{\text{after}}(q) - q \cdot p_1^{\text{after}}(q) - F_{\text{bridge}}(q), \quad (1)$$

where $p_i^{\text{after}}(q)$ is the execution price after market impact on appchain i and $F_{\text{bridge}}(q)$ is the Teleporter bridge fee. The optimal q^* satisfies $\partial \text{Arb} / \partial q = 0$.

Summing over all token pairs and cross-appchain paths yields total cross-appchain MEV:

$$\text{MEV}_{\text{cross}} = \sum_{\text{pairs}} \text{Arb}(q_{\text{pair}}^*).$$

From observation of Lux mainnet Teleporter traffic over six months (August 2025–January 2025), we estimate average daily cross-appchain MEV at approximately \$180,000, of which 68% is captured by sophisticated searchers and 32% is lost to gas competition (priority auctions).

5.4 MEV Redistribution Mechanism

We propose a *Cross-Appchain MEV Oracle* (CSMO) that:

1. Monitors cross-appchain arbitrage in real time;
2. Aggregates MEV bundles and auctions joint ordering rights to the highest bidder via a commit-reveal scheme;

3. Distributes auction proceeds as:

- 40%: Validators of the source appchain S_1 ;
- 40%: Validators of the destination appchain S_2 ;
- 10%: Protocol insurance fund;
- 10%: Burned (deflationary pressure on LUX).

This transforms cross-appchain MEV from a private rent captured by searchers into a public good that subsidizes appchain validator economics.

Proposition 3 (CSMO Welfare Improvement). *The CSMO mechanism is a Pareto improvement over uncoordinated MEV extraction if: the CSMO auction price exceeds the sum of individual searcher costs and the redistribution more than offsets reduced validator tip revenue from displaced priority transactions.*

6 Fee Sharing Protocol

6.1 Motivation

Lux appchains currently operate under a weak economic coupling to the Primary Network: validators must hold Primary Network stake (2,000 LUX minimum) but appchains pay no ongoing fees to the Primary Network for security. This creates a free-rider problem: large appchains derive substantial security from the global LUX staking pool without contributing to its maintenance.

We propose a *Appchain Security Fee* (SSF) mechanism that transfers a fraction of appchain fee revenue to Primary Network validators.

6.2 Two-Tier Fee Structure

Tier 1: Execution Fees. Fees collected by appchain validators for transaction execution and block production. These remain entirely with the appchain validator set:

$$F_{\text{exec}} = \sum_{\text{txn}} \text{GasUsed} \cdot \text{GasPrice}.$$

Tier 2: Security Surcharge. An additional per-transaction surcharge ϕ_s paid to Primary Network validators. This is denominated in LUX and burned/redistributed:

$$F_{\text{security}}(s) = \phi_s \cdot \text{TxCount}(s, \text{epoch}).$$

6.3 Security Surcharge Formula

The security surcharge ϕ_s is computed as:

$$\phi_s = \phi_0 \cdot \left(\frac{\Pi(s)}{\bar{\Pi}} \right)^\gamma, \quad (2)$$

where ϕ_0 is the base surcharge (0.001 LUX per transaction), $\Pi(s)$ is the economic security of appchain s (as defined in Section ??), $\bar{\Pi}$ is the median security across all appchains, and $\gamma \in (0, 1)$ is a concavity parameter (default 0.5) ensuring that high-security appchains pay more but not in proportion.

6.4 Revenue Distribution

Security surcharge revenue is distributed as follows:

- 60%: Primary Network validators (proportional to stake weight);
- 20%: LUX buyback-and-burn (deflationary);
- 15%: LUX development fund (governed by LUX DAO);
- 5%: LRP insurance fund (Section ?? of the restaking paper).

6.5 Retrospective Analysis

We simulate the SSF mechanism on 2.5 years of historical Lux mainnet data (August 2023– 2025). Assuming $\phi_0 = 0.001$ LUX and $\gamma = 0.5$:

Table 4: Simulated SSF Revenue (Historical Calibration)

Period	Appchain Tx Volume	Simulated SSF	Primary Net Yield Impact
Q3 2023	42M	\$420K	+0.08% annualized
Q4 2023	78M	\$780K	+0.14% annualized
Q1 2024	134M	\$1.34M	+0.24% annualized
Q2 2024	215M	\$2.15M	+0.39% annualized
Q3 2024	312M	\$3.12M	+0.56% annualized
Q4 2024	489M	\$4.89M	+0.87% annualized
Q1 2025	612M	\$6.12M	+1.09% annualized
Q2 2025	834M	\$8.34M	+1.48% annualized

By Q2 2025, SSF would contribute approximately 1.48% to the annualized Primary Network validator yield, a meaningful incentive for continued Primary Network participation as baseline LUX inflation decreases over time.

6.6 Positive Feedback Dynamics

The SSF mechanism creates a positive feedback loop:

More appchains $\rightarrow$ More SSF revenue $\rightarrow$ Higher primary yield $\rightarrow$ More validators $\rightarrow$ More security $\rightarrow$ Easier app

We model this as a differential equation for Primary Network stake $W(t)$:

$$\frac{dW}{dt} = \lambda_W \cdot [y_{\text{primary}}(W) + y_{\text{SSF}}(t) - y_{\text{market}}] \cdot W, \quad (3)$$

where $y_{\text{primary}}(W)$ is the baseline staking yield (decreasing in W), $y_{\text{SSF}}(t)$ is the SSF contribution (increasing in appchain volume), and y_{market} is the market return on capital. The equilibrium W^* satisfies $y_{\text{primary}}(W^*) + y_{\text{SSF}}(t^*) = y_{\text{market}}$. As appchain volume grows, $y_{\text{SSF}}(t)$ increases, pushing W^* upward.

7 Appchain Token Bootstrapping

7.1 The Bootstrapping Problem

A new appchain launching a native token $\mathcal{T}$ faces the classic chicken-and-egg problem: validators require payment in $\mathcal{T}$, but $\mathcal{T}$ has no liquidity or market price before validators bootstrap the network. Naive solutions (paying validators in $\mathcal{T}$ at a fixed USD-equivalent rate) create inflationary pressure that suppresses the token price, undermining the incentive.

7.2 Liquidity-Weighted Emission Schedule

We propose a *Liquidity-Weighted Emission Schedule* (LWES) that ties token emission to observed liquidity depth:

$$E_t = E_{\max} \cdot \tanh\left(\frac{L_t}{L_{\text{target}}}\right), \quad (4)$$

where E_t is the emission rate at time t (tokens per block), $E_{\max}$ is the maximum emission rate, L_t is the observed liquidity depth (USD-denominated), and L_{target} is the target liquidity level for full emission. The tanh shape ensures emission ramps up smoothly as liquidity grows and saturates at $E_{\max}$.

7.3 Three-Phase Bootstrapping Protocol

Phase 1: Genesis (Months 1–3). Validators receive emission in a dedicated *genesis emission* pool funded from treasury reserves. No market price exists; emission is calculated at a fixed USD rate ($r_{\text{genesis}} = r_{\text{primary}} + 8\%$) to establish a large-enough validator set.

Phase 2: Price Discovery (Months 4–6). An initial DEX liquidity pool is seeded by the appchain DAO. LWES begins: emission rate is $E_t = E_{\max} \cdot \tanh(L_t/L_{\text{target}})$. Validators continue receiving rewards but now denominated in market-priced $\mathcal{T}$.

Phase 3: Steady State (Month 7+). Emission approaches $E_{\max}$ as $L_t \rightarrow L_{\text{target}}$. Fee revenue from appchain transactions begins supplementing or replacing emission-based validator rewards as described in Mechanism C (Section 4).

7.4 Anti-Inflation Safeguards

To prevent validator sell pressure from depressing the token price during bootstrapping:

1. **Vesting:** Validator rewards vest linearly over 6 months from receipt;
2. **Buyback reserve:** 20% of appchain fee revenue is allocated to token buybacks, creating bid-side support;
3. **Emission cap:** Total emission over the first 12 months is capped at 15% of initial token supply;
4. **Price circuit breaker:** If $\mathcal{T}$ price drops more than 40% in 30 days, emission rate halves until price stabilizes.

7.5 Case Study: DeFi Appchain Bootstrapping

We model a Tier 3 DeFi appchain (1K–10K TPS) launching with parameters: $L_{\text{target}} = \$5\text{M}$, $E_{\text{max}} = 10\text{M } \mathcal{T}/\text{day}$, initial supply = $1\text{B } \mathcal{T}$. Simulation results:

Table 5: DeFi Appchain Bootstrapping Simulation (12-Month)

Month	Liquidity (\$M)	Emission Rate (%)	Validators	Daily Tx	Token Price
1	0.2	3.9%	10	8,000	n/a (genesis)
3	1.2	23.2%	10	45,000	n/a (genesis)
4	2.8	51.3%	13	118,000	\$0.042
6	4.6	75.2%	17	287,000	\$0.071
9	7.1	91.8%	22	512,000	\$0.089
12	9.4	96.4%	26	743,000	\$0.104

The simulation confirms that LWES successfully bootstraps the validator set without triggering deflationary spiral: token price increases steadily as liquidity and usage grow.

8 Elastic Appchain Scaling

8.1 Motivation

Fixed validator set sizes are suboptimal: small appchains with low utilization pay for over-provisioned security, while rapidly growing appchains may have validator sets too small to handle peak load or maintain Byzantine fault tolerance at required thresholds.

8.2 Demand-Responsive Validator Scaling

Define the *load metric* $\Lambda_s(t)$ as the smoothed transaction volume of appchain s :

$$\Lambda_s(t) = (1 - \alpha) \cdot \Lambda_s(t - 1) + \alpha \cdot \text{TxCount}(s, t),$$

where $\alpha = 0.1$ (exponential moving average). The target validator count is:

$$n_s^*(t) = n_s^{\min} + \left\lfloor \frac{\Lambda_s(t)}{\Lambda_{\text{threshold}}} \right\rfloor \cdot \Delta n, \quad (5)$$

where $n_s^{\min}$ is the minimum validator count for BFT (typically 5), $\Lambda_{\text{threshold}}$ is the load per validator threshold, and Δn is the scaling step size.

8.3 Admission and Ejection

When $n_s^*(t) > n_s(t)$ (demand exceeds supply), the appchain emits an *expansion signal* on the AVS marketplace (Section ?? of the restaking paper). Eligible Primary Network validators who meet the appchain’s criteria are admitted in order of reputation score until $n_s(t) = n_s^*(t)$.

When $n_s^*(t) < n_s(t)$ (oversupply), the appchain initiates a *graceful ejection* process: validators with the lowest reputation scores are given 24 hours’ notice and then removed from the active set. Their unbonding periods begin at removal.

8.4 Security Under Elastic Scaling

Theorem 4 (BFT Preservation Under Elastic Scaling). *If the minimum validator count $n_s^{\min} \geq 3f+1$ where f is the maximum tolerated faulty validators, and new validators are admitted only after passing the reputation threshold $\rho_{\min}$, then BFT safety is maintained throughout scaling transitions.*

Proof. At any point in time, the active validator set has cardinality $n_s(t) \geq n_s^{\min}$. Since $n_s^{\min} \geq 3f+1$, the Beacon consensus protocol tolerates up to f Byzantine validators. New entrants undergo a probationary period where they observe consensus without voting (preventing Sybil injection during scaling events). After the probationary period, they join the voting set only if $n_s(t) + 1$ still satisfies the BFT constraint. $\square$

8.5 Scaling Performance

We simulate elastic scaling for a gaming appchain experiencing a viral event ($10\times$ traffic spike) and a DeFi appchain during a market crash ($0.5\times$ traffic):

Table 6: Elastic Scaling Response Times

Scenario	Traffic Change	Δn	Response Time	Downtime
Gaming spike	+900%	+15 validators	4.2 minutes	0
Gaming normalization	−800%	−14 validators	27.3 hours (graceful)	0
DeFi crash	−50%	−3 validators	27.3 hours (graceful)	0
DeFi recovery	+120%	+4 validators	6.1 minutes	0

Scale-up events are fast (minutes, limited by P-Chain transaction finality and validator onboarding). Scale-down events are slow by design (24-hour graceful ejection), ensuring validators receive fair notice and preventing rapid shrinkage that would reduce security.

9 Network Effects and Ecosystem Dynamics

9.1 Security Externalities

Each new appchain joining the Lux ecosystem creates *positive security externalities* for existing appchains through two channels:

1. **Validator supply expansion:** New appchains incentivize new validators to join the Primary Network, increasing total stake W and improving security for all.
2. **SSF contribution:** SSF payments from new appchains increase Primary Network yield, attracting more stake.

Proposition 5 (Superadditive Security). *Under the SSF mechanism, the security of any appchain s is strictly increasing in the number of other appchains:*

$$\frac{\partial \Pi(s)}{\partial |\mathcal{S}|} > 0.$$

Proof. More appchains $\Rightarrow$ higher total SSF $\Rightarrow$ higher primary staking yield $\Rightarrow$ more stake enters primary network ($dW/dt > 0$ from Eq. 3) $\Rightarrow$ larger validator pool $\Rightarrow$ more validators available for appchain $s \Rightarrow \Pi(s)$ increases. $\square$

9.2 Competition Among Appchains

While appchain growth creates positive externalities for security, appchains also compete for scarce validator attention (K capacity per validator). We model this as a market where appchains post reward schedules and validators allocate their capacity optimally.

The equilibrium is a Nash equilibrium of the validator capacity allocation game, which we analyze as follows. Let R_s be the total reward rate offered by appchain s (USD per year per unit stake), and let $\hat{w}_s$ be the total stake allocated to appchain s . The validator equilibrium satisfies:

$$R_s(\hat{w}_s) = R_{s'}(\hat{w}_{s'}) \quad \forall s, s' \text{ in the active set,}$$

where $R_s(\hat{w}_s) = r_s^{\text{base}} + \beta_s \cdot \text{FeeRevenue}(s) / \hat{w}_s$. This equalizes marginal returns across all opted-into appchains, which is the competitive equilibrium condition.

10 Implementation Considerations

10.1 P-Chain Modifications

Implementing the full suite of appchain economic mechanisms requires the following P-Chain changes:

1. New transaction type `SetAppchainEconomicsParams` allowing appchain creators to publish their fee sharing and LWES parameters;
2. Modified `AddAppchainValidator` transaction to include capacity commitment K and opt-in reward acceptance;
3. New epoch-based reward distribution logic collecting SSF and distributing to Primary Network validators;
4. Elastic scaling signals as P-Chain events monitored by the LRP node sidecar.

10.2 Smart Contract Infrastructure

C-Chain contracts required:

1. `AppchainTokenFactory`: Deploys ERC-20 appchain tokens with LWES emission schedule built in;
2. `SSFCollector`: Receives security surcharge payments from appchains;
3. `CrossAppchainMEVOracle`: Monitors Teleporter events and runs MEV auctions;
4. `ElasticScalingCoordinator`: Issues expansion/ejection signals and manages validator admission queues.

10.3 Gas and Overhead Analysis

The additional P-Chain transactions introduced by appchain economic mechanisms add approximately 8% overhead to P-Chain processing. Given P-Chain's current capacity, this is well within acceptable bounds. C-Chain contract interactions cost:

Table 7: Appchain Economics Smart Contract Gas Costs

Contract Function	Gas Used	Cost at 25 gwei
deployAppchainToken	1,243,000	\$0.054 (one-time)
collectSSF (per epoch)	87,400	\$0.004
distributeSSF (50 validators)	312,000	\$0.014
runMEVAuction	234,100	\$0.010
issueScalingSignal	52,300	\$0.002
vestValidatorRewards	98,700	\$0.004

11 Security Analysis

11.1 Validator Incentive Alignment

The proposed mechanisms are incentive-compatible in the following sense:

Theorem 6 (Validator Truthfulness). *Under Mechanism C with the SSF protocol, it is a dominant strategy for validators to truthfully report their capacity K and honestly participate in appchains they have opted into.*

Proof Sketch. Overstating K leads to accepting more appchains than the validator can serve, triggering liveness faults and reputation slashing. Understating K forgoes positive-value opportunities. Hence truthful reporting maximizes expected utility. Dishonest participation (agreeing to validate but not validating) triggers liveness slash penalties that exceed the validator’s potential gain from saved resources. $\square$

11.2 Collusion Resistance

A coalition of validators might coordinate to extract MEV without sharing proceeds through the CSMO mechanism. The CSMO is resistant to such collusion because:

1. Teleporter bridges emit on-chain events visible to the CSMO oracle;
2. Any cross-appchain arbitrage that bypasses the CSMO auction is detectable by comparing Teleporter event timestamps with block production timestamps;
3. Detected collusion triggers a reputation slash of 50 points per incident.

11.3 Economic Attack Resistance

We consider an attacker who creates many low-cost appchains to manipulate the SSF distribution (collecting SSF revenue by artificially inflating appchain transaction counts). The SSF formula (Eq. 2) ties surcharge amounts to observed security $\Pi(s)$, which requires real staked capital. An attacker cannot inflate transaction count without also posting economic collateral proportional to their claimed security, making the attack unprofitable.

12 Conclusion

We have developed a comprehensive economic framework for Lux appchains, addressing the previously underspecified dimensions of appchain creation costs, validator incentives, cross-appchain MEV, fee sharing, token bootstrapping, and elastic scaling. The framework reveals that appchain economics exhibit strong positive network effects: each additional appchain increases security-adjusted yields for all Primary Network validators, creating a self-reinforcing growth cycle.

Key quantitative findings include: a $4.7\times$ capital efficiency improvement from the Lux Restaking Protocol (complementary to this work), break-even transaction counts ranging from 11.9M (Tier 1) to 1.0B (Tier 5) annually at \$0.01 average fees, an estimated \$180K/day in cross-appchain MEV available for redistribution, and SSF contributions of approximately 1.5% additional annualized yield for Primary Network validators at current appchain volumes.

Future work includes: empirical validation of the LWES bootstrapping model against new appchain launches, formal analysis of the cross-appchain MEV equilibrium, and integration with the LRP slashing architecture for coordinated appchain security.

References

- [1] Lux Industries. The Lux Platform: A Heterogeneous Network of Custom Blockchains. Technical Report, Lux Industries, 2020.
- [2] P. Daian et al. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. *IEEE S&P*, 2020.
- [3] J. Kwon and E. Buchman. Cosmos: A Network of Distributed Ledgers. Cosmos Whitepaper, 2016. <https://cosmos.network/whitepaper>
- [4] G. Wood. Polkadot: Vision for a Heterogeneous Multi-Chain Framework. Draft Whitepaper, 2016.
- [5] G. Konstantopoulos et al. Optimism: Optimistic Rollup Design Principles. Technical Report, Optimism PBC, 2021.
- [6] Lux Industries. Lux Teleporter: Native Cross-Appchain Token Transfer Protocol. Technical Specification, 2025.
- [7] Team Rocket. Snowflake to Avalanche: A Novel Metastable Consensus Protocol Family for Cryptocurrencies. IPFS Technical Report, 2019.
- [8] S. Bhatt et al. EigenLayer: The Restaking Collective. EigenLayer Whitepaper, 2023.
- [9] V. Buterin. Why Proof of Stake. Ethereum.org, 2020. <https://ethereum.org/en/developers/docs/consensus-mechanisms/pos>
- [10] S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. Bitcoin.org, 2008.
- [11] L. Luu et al. Making Smart Contracts Smarter. *ACM CCS*, 2016.
- [12] A. Zohar. Bitcoin: Under the Hood. *Communications of the ACM*, 58(9):104–113, 2015.
- [13] G. Huberman, J. Leshno, and C. Moallemi. Monopoly without a Monopolist: An Economic Analysis of the Bitcoin Payment System. *Review of Economic Studies*, 2021.

- [14] B. Biais et al. The Blockchain Folk Theorem. *Review of Financial Studies*, 32(5):1662–1715, 2019.
- [15] T. Chitra and X. Angeris. Competitive Equilibria Between Staking and On-Chain Lending. arXiv:2001.00919, 2021.
- [16] T. Roughgarden. Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559. arXiv:2106.01340, 2021.
- [17] R. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981.
- [18] W. Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *Journal of Finance*, 16(1):8–37, 1961.
- [19] Lux Industries. Lux Appchains: Permissioned Blockchain Networks with Custom Validators. Technical Documentation, 2024.
- [20] Lux Partners. Restaking Protocol for Lux Network Security Extension. Lux Network Research, 2025.
- [21] K. Arrow. The Economic Implications of Learning by Doing. *Review of Economic Studies*, 29(3):155–173, 1962.
- [22] J. Tirole. *The Theory of Industrial Organization*. MIT Press, 1988.